Data Structure & Algorithm Design

Optimizing Storage and Process Logic for Modern Systems

// PART 01

Data Storage Design

Structuring data for speed, efficiency, and scalability.

Core Objectives

Fast

Rapid retrieval and processing of information to minimize latency.

Efficient

Optimized use of storage space and computing resources.

Accessible

Seamless data availability for authorized users and applications.

Maintainable

Easy to update, backup, and manage over the system lifecycle.

The Storage Dilemma: Files vs. Databases

A. Files

Simple lists of records saved directly to the OS.

Master File: Permanent data (e.g., Customers).

Look-up File: Fixed lists (e.g., Country codes).

Transaction File: Temp updates (e.g., Daily sales).

⚠ *Pros: Simple, Cheap. Cons: No relationships, Not scalable.*

Database Architectures

Relational (RDBMS)

Most common. Uses **tables**, primary/foreign keys, and SQL. Ideal for structured transaction data.

Object-Oriented

Designed for complex data like images/video. Works with objects, classes, and inheritance.

Multidimensional

Used for Data Warehousing & Analytics (DSS). Allows "slicing and dicing" for decision support.

Legacy

Hierarchical or Network models. Fast but rigid and hard to update. Rarely used in new systems.

Selecting a Format

Data Type: Simple → Relational; Multimedia → Object; Analytics → Multidimensional.

System Type: Transaction Processing (high updates) vs. Decision Support (read-only).

Existing Systems: Align with current stack to reduce learning costs.

Future Needs: Choose widely supported, scalable technologies.

From Logical to Physical Models

Converting the business view (Logical ERD) into the implementation view (Physical ERD).



Entities
become Tables



Attributes
become Fields



Keys
Add Primary & Foreign



System Details
Indexes & Audit Logs

// PART 02

Algorithm & Program Design

Defining logic, modules, and system behavior.

Refining the Process

Transforming a Logical DFD into a Physical DFD requires adding concrete implementation details.

- **Add Tech Details:** Specify filenames, DB tables, and validation steps.
- **Human-Machine Boundary:** Separate manual tasks from automated ones.
- **System Processes:** Add error handling, logging, and authentication.
- **Update Metadata:** Record types, sizes, and timing in the CASE repository.

Modular Program Design

Programs must be modular, clear, and easy to maintain. The primary tool for this is the **Structure Chart**.

Structure Chart Purpose:

Visually represents the top-down organization of a program, showing independent modules and how data passes between them.

Chart Components

Modules

Control Modules manage flow.
Subordinate Modules perform specific tasks.

Connections

Control Links: Who calls whom.
Data Couples: Data passed.
Control Couples: Flags/Messages.

Logic Patterns

Sequence: Order of execution.
Selection: If/Else conditions.
Iteration: Loops.

Building a Structure Chart

01

Identify Modules

From DFD Processes

02

Organize

General → Specific

03

Add Logic

Loops & Decisions

04

Add Couples

Data & Control

05

Review

Check Cohesion

"High Cohesion, Low Coupling"

The Golden Rule of Modular Design

Modules should do **one thing well** (cohesion) and be as **independent** as possible (coupling). A good design also promotes high reuse (fan-in) and avoids micromanagement (high fan-out).

The Final Output: Program Specification

A detailed guide ensuring programmers know exactly what to build.

- **Program Info:** Name, Developer, Deadline.
- **Trigger Events:** What starts the program?
- **Inputs & Outputs:** Data entry and results.
- **Pseudocode:** Step-by-step logic instructions.

Ready for Implementation